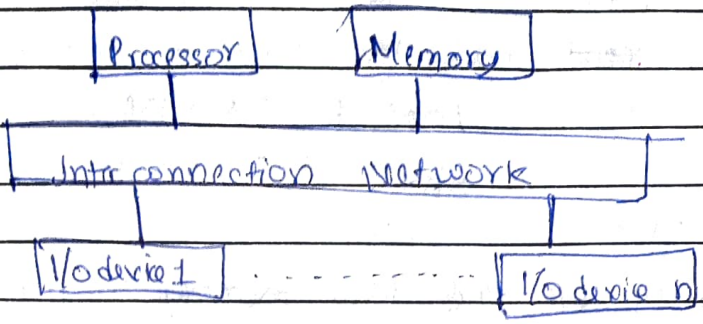
DATE: PAGE:

# Basic Input Output

## Accessing I/O Devices




* Components of a computer communicate with each other with help of interconnection networks to Busses. (set of wires which are used to connect internal components of computer)
* 3 dedicated lines are available in every bus → Address Line
* Each I/O device is assigned a unique → Control line
address. → Data line
* Some addresses in address space of processor are assigned to to these I/O locations. These locations are usually implimented as flip flops (which can store 1-Bit) organised in the form of registers → I/O registers. If the main memory and I/O devices share the same address space → Memory-Mapped I/O → used in many computers.
* In other words, if same address space is used for both main memory and I/O it is called Memory mapped I/O and if separate address space is used for I/O and main memory it is called I/O mapped I/O.
* On MMIO, I/O devices are treated as normal memory locations

DATE: PAGE:

## Memory mapped I/O

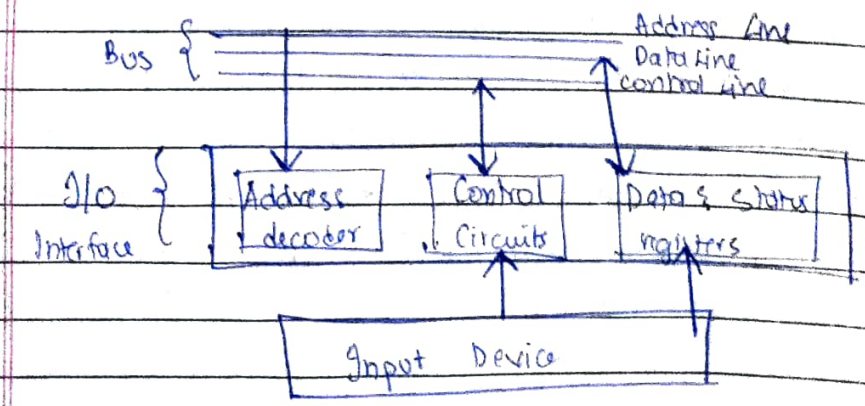
* Address of both I/O device and main memory is mapped to same address space
* I/O device treated as main memory
* Normal ~~read~~ load, store operations are used as instructions
  Load R2, DATA IN
  Store R2, DATA OUT

## I/o mapped I/o

* Address of I/o and main memory are mapped to different address space.
* Special instructions to transfer data ~~from~~ to & from I/O device. Like IN, OUT etc.
* Separate busses are not used for memory and I/o device
* ~~Both~~, address lines (in Bussed) may be shared b/w I/O devic and memory, with a control signal to indicate whether it is a memory address or I/O address

## I/o device Interface

* An I/o device is connected to interconnection network by using a circuit called the divice Interface.



DATE: PAGE:

* I/o device is connected to Bus using I/O interface circuit which has Address decoder, control circuits and Data & status registers.
* Address Decoder → Decodes the address placed on address lines thus enabling the device to recognise address.
* Data register → Holds data being transfered to or from the processor
* Status register → Holds information necessary for operation of I/o device.
* Data and status registers are connected to data lines, and have unique addresses.
* Control circuits → Helps coordinate b/w all operations

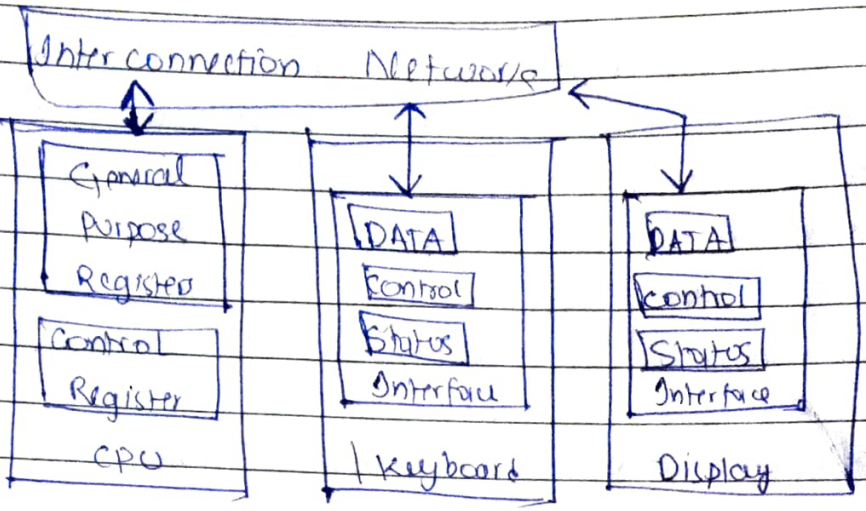
• Mechanism s used for synchronising data transfer b/w processor and memory

The difference in speed (data transfer) b/w processor & I/O device creates need for mechanism to synchronize the transfer of data b/w them.

1) Program-controlled I/o
2) Interrupts
3) Direct Memory Access

[1] Program controlled I/o

• Here processor program controls the complete data transfer Only When I/o transfer instruction is executed, the transfer can take place. First it checks if the device is ready / not for data transfer (using status reg) on then the data is transferred.




Connection for processor and display

- **Keyboard**

Task → Reads characters from keyboard ⟶ Stores in Memory → Displays the same character

- Solution → SIGNALING PROTOCOL

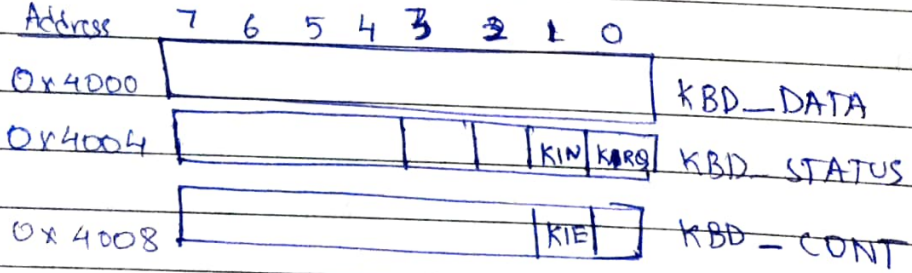



Fig: Keyboard Interface

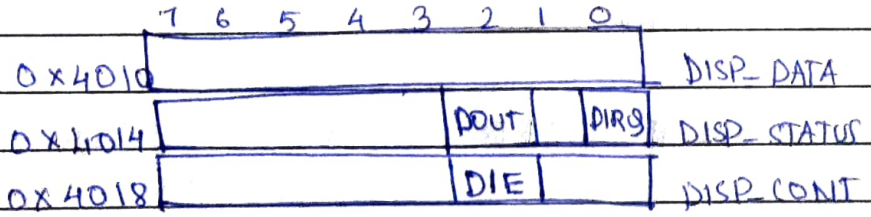
- The processor waits for a signal from key board if it ready to give input. An input character is obtained from the keyboard. The processor waits for a signal that key has been pressed and binary code (ASCII value) is stored ~~in Data register~~ in I/O register and then it reads the binary code from it. In a similar way, processor waits for a signal from display that a character can

be sent. It sends the character and the process goes on.

* Let
  KBD_DATA → Address of 8-bit Data register of keyboard interface which holds character
  KBD_STATUS → Address of 8-bit status register
  KBD_CONT → Address of 8-bit control register

* Signal indicating key has been pressed → KIN in KBD_STATUS → Setting to 1. (called and stored in KBD_DATA; KIN is a flip flop)
  The processor reads the status flag KIN to determine whether character is placed in KBD_DATA.

* When processor reads status flag (It is called) → Processor polls the I/O device.




Display Interface

* DISP_DATA → Address of 8 bit display data register
  DISP_STATUS → Address of 8-bit status register
  DISP_CONT → Address of 8 bit control register
  } of display interface
* DISP_DATA → Recieves character from processor
  DOUT → Status flag → gives signal to processor that display is ready to recieve next character.

Steps in key board input

1) Key is pressed
2) i) Keyboard places ASCII value of key in KBD_DATA
   ii) KIN of KBD_STATUS is set to 1

3.i) Processor checks for KIN, if it is set to 1, it transfers content of KBD_DATA to processor register.
ii) KIN is set to 0.
4.) Cycle continues

READWAIT { Read the KIN flag
Branch to READWAIT if KIN = 0
Transfer data from KBD_DATA to R5

(Keeps checking for status flag.)

Display

1) i) Processor keeps checking on DOUT, when DOUT = 1, the processor transfers character to DISP_DATA
ii) DOUT = 0
2) Cycle continues

WRITEWAIT Read DOUT flag
Branch to WRITEWAIT if DOUT = 1
Transfer data from R5 to DISP_DATA

* Initially KIN=0 & DOUT = 1

In Memory-Mapped I/O

READWAIT: LoadByte R4, KBD_STATUS
And R4, R4, #2
Branch if [R4]=0 READWAIT
LoadByte R5, KBD DATA

(To check for flag which is in bit 1 in KBD_STATUS)

WRITEWAID: LoadByte R4, DISP_STATUS
And R4, R4, #4
Branch if [R4]=0 WRITEWAIT
StoreByte R5, DISP_DATA

DATE: PAGE:

RISC style program that reads char and displays it

```
        Move        R2, #LOC        Initialize pointer register R2 to point to address of first location in memory where characters are to be stored
        MoveByte    R3, #CR         Load ASCII code for carriage return to R3
READ:   LoadByte    R4, KBD_STATUS  Wait for char to be entered.
        AND         R4, R4, #2      Check KIN flag
        Branch if_[R4]=0 READ
        LoadByte    R5, KBD_DATA    Read character from KBD_DATA (clears KIN to 0)
        StoreByte   R5, (R2)        Write the character into main memory
        Add         R2, R2, #1      and increment the pointer.
ECHO:   LoadByte    R4, DISP_STATUS Wait for display to become ready
        AND         R4, R4, #4      check for DOUT flag
        Branch_if_[R4]=0 ECHO
        StoreByte   R5, DISP_DATA   Move the character just read to display buffer register (DOUT=0)
        Branch_if_[R5]≠[R3] READ    Check if character just read is carriage return. If not, then branch back to read another character.
```

> carriage return (CR) is a character moves the cursor to the beginning of the line without advancing to next line

DISK programs

Test Bit destination, #k → checks if kth bit ($b_k$) of 'destination' operand and sets the condition flag (Z) to 1 if $b_k = 0$ and to 0 if $b_k = 1$.

DATE: PAGE:

Compare destination, source → Performs comparision by subtraction (dest − sou) and sets condition flag based on it. It doesn't alter dest & sour

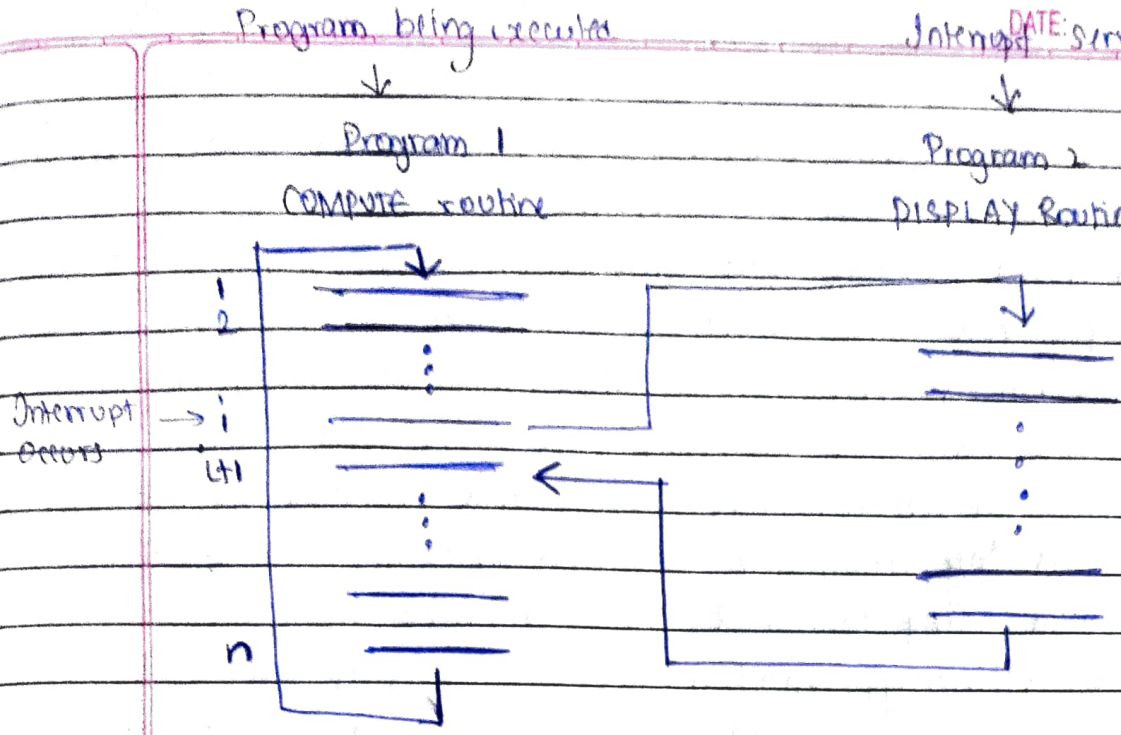
```
        Move        R2, #LOC
READ:   Test Bit    KBD_STATUS, #1
        Branch =0   READ
        MoveByte    (R2), KBD_DATA

ECHO:   Test Bit    DISP_STATUS, #4
        Branch =0   ECHO
        Move Byte   DISP_DATA, (R2)

        CompareByte (R2)+, #CR      Check if char just read is CR
        Branch ≠0   READ.           If not then branch back to
```

- **INTERRUPT**

* We may note that in Program controlled I/O, processor continiously executes/goes through WAIT cycle (i.e for KIN or DOUT to be 1) which is complete time waste. Instead of that, if processor gets a signal when the KIN or DOUT is ready and during this time (WAIT cycle), processor can execute some other tasks.
* The hardware signal sent to processor when I/O device is ready → Interrupt signal → Interrupt-request line (one of control lines in BUS) is dedicated for this purpose.




* Let us assume that interrupt request arrives during execution of instruction 'i' of program 1.
  1. Instruction at location 'i' is executed
  2. The contents of PC (i+1), condition code flags, processor registers used by both interrupted program and interrupt service routine is stored and Execution of Prog 1 → Suspended
  3. Control is transferred to first instruction of Program 2
  4. After execution of interrupt service routine, the contents of processor (during execution of program 1) is reloaded and control is transfered to (i+1).
  5. The return address (i+1) is stored either in register or processor stack

* The processor must inform the I/O device that its request has be recognised so that it removes its interrupt-request signal. This is done by control signal called INTERRUPT ACKNOWLEDGE which is sent to I/O device through Interconnection network.
On the other case, the data transfer that takes place between I/O is itself used as acknowledgement.

* Saving and restoring registers involves memory transfer
  → Increases total execution time
  → Increase delay b/w time an interrupt request is received and start of execution of interrupt service routine. This is called INTERRUPT LATENCY
* To reduce this, most processors save only minimal amount of information i.e PC and processor status registers. Any additional information to be saved must be saved using additional program instructions.
* To avoid Latency, another set of registers can be used by interrupt-service routine eliminating the need to save processor state. This is called SHADOW REGISTERS

- ~~Enabling and Disabling Interrupts~~

* Interrupts can arrive at any time in a program which may alter the sequence of events written by programmer. Hence is is convinient to enable and disable interrupts at both processor and I/O device ends. The processor can either accept or deny interrupt request.
* Processor has status register (PS) which has info about current state of operation. One bit, IE, of this register be assigned for enabling/disabling interrupt. The I/O device is allowed to raise interrupt request only when this bit (IE) is set to 1.

- To avoid interruption cycle by same device during execution of an interrupt cycle ~~can be done by~~
  1. First instruction of interrupt service routine must be ~~disabled~~ Interrupt disabled

DATE: PAGE:

2. Last instruction must be, ~~disabled~~ ~~Interrupt~~ Interrupt enabled.

2. Last instruction must be, ~~disabled~~ Interrupt enabled.

Interrupt Cycle

1. Device raises an interrupt request
2. The processor interrupts the program currently being executed and saves the contents of PC and PS registers.
3. Interrupts are disabled by clearing IF bit in PS to 0.
4. The action requested by interrupt is performed by interrupt service routine, during which the device is informed that its request has been processed, and in response, it deactivates the interrupt-request signal.
5. Upon completion of interrupt-service-routine, the saved contents of PC and PS registers are restored (enabling interrupts by setting IF to 1), and execution of interrupted program is resumed.

- Handling Multiple Devices

* The information needed to determine whether a device is requesting an interrupt is available in IRQ bit of status register. IRQ is set to one if a device raises interrupt request
* Interrupt service routine polls the I/O device connected to the bus. The first device with IRQ equal to 1 is the one that is served
* This polling mechanism is time consuming to keep checking IRQ for IRQ bit of all I/O devices connected to bus.
* Instead vectored interrupts are used

DATE: PAGE:

- Vectored Interrupts
  - The device raising interrupt may identify itself directly to the processor.
  - A device requesting an interrupt can identify itself if it has its own interrupt-request signal or if it can send a special code to processor. Code supplied by the device represents part of starting address of interrupt-service routine.
  - A commonly used scheme is to allocate permanently an area in memory to interrupt vectors, and they are said to contribute interrupt-vector table. (Interrupt vector → address of interrupt-service-routines). The interrupt-service routine can be located anywhere in memory.
  - When an interrupt request arrives, the information provided by requesting device is used as pointer into interrupt-vector table and address in corresponding vector is automatically loaded to PC and corresponding interrupt-service-routine is executed.

- Interrupt Nesting
  - Till an interrupt request is denied when an interrupt service is already being executed, but at times we require interrupt nesting i.e to accept interrupt request even if some interrupt-service-routine is being executed.
  - This suggests that I/O devices should be arranged in a priority structure. An interrupt request from higher priority device should be accepted while processor is servicing a request from a lower priority device.

DATE: PAGE:

* The interrupted interrupt-service routine (of low priority device) will continue execution after interrupt-service routine of higher-priority divice is executed.

- Simultanious Interrupts

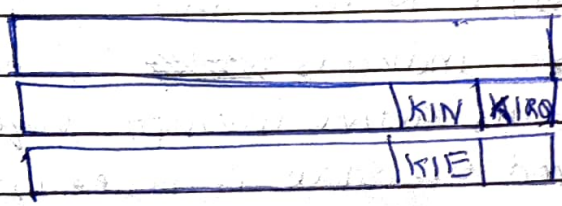
When simultanious interrupt requests arrive, processor must decide which request to serve first.

* Polling the status register is simplest mechanism. In such case, priority is determined by order in which devices are polled.
* When vectored interrupts are used, it is to be made sure that only one device sends interrupt vector code. This is done by arbitration circuits.

- Controlling I/O device behavior

* To ensure that interrupt requests are generated only by those I/O devices that the processor is currently willing to recognise, a mechanism is needed in the interface circuits of individual devices.

*



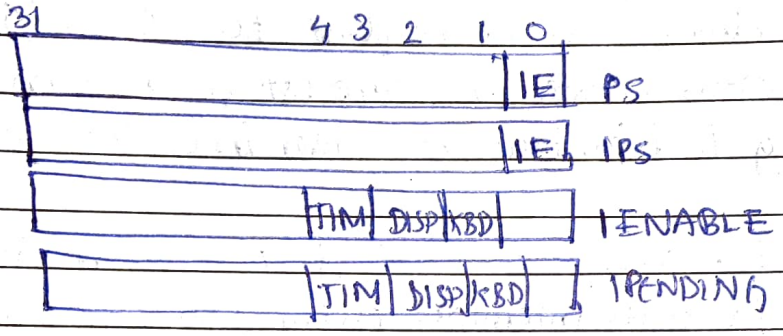

eg→

KIE → Interrupt enabled ⇒ keyboard can send interrupt request
KIRQ=1 → Interrupt has been send but not yet processed by processor
KIN → 1 → If input is set in KBD_DATA

DATE: PAGE:

* Interrupt req can be raised only when KIE=1 and KIN=1.

- Processor Control Registers




- PS → Status register reads IE bit of control register in interface
- IPS → Automatically saves the contents of PS when an interrupt request is accepted. After interrupt-service routine is executed, contents of IPS are transfered to PS restoring the processor.
- IENABLE → Allows processor to selectively respond to I/O devices. A bit is assigned to each device. When a bit is set to 1, register indicates interrupt request from (corresponding) device.
- IPENDING → Indicates active interrupt request.

* Contents of IPS saved on stack if nested interrupts are allowed

* These program control registers cannot be accessed in same way as general purpose registers can be accessed. Because they use 5-bit field issued to specify destination and source registers.

DATE: PAGE:

MoveControl R2, PS

MoveControl IENABLE, R3

- **Exceptions**

* The term exception is used to refer to any event that causes interruption.

Eg → 1) Interrupt - request from I/O devie is one type of exception
2) Recovery from error
3) Debugging
4) Privilage exception

- **Recovering from error**

* There may be many sources of error
→ Error in data stored
→ Error during execution of an instruction

* When such errors are detected, exception processing is initiated.
→ Processor takes same steps as in case of I/O interrupt request
→ It suspends execution of current program, and starts executing exception-service routine.

* But a small difference I/O interrupt & exception service routine
→ In I/O interupt, current instruction being executed by processor is executed and then interrupt service routine is executed while ~~in exception~~ in exception processing, current instruction cannot be executed completely.

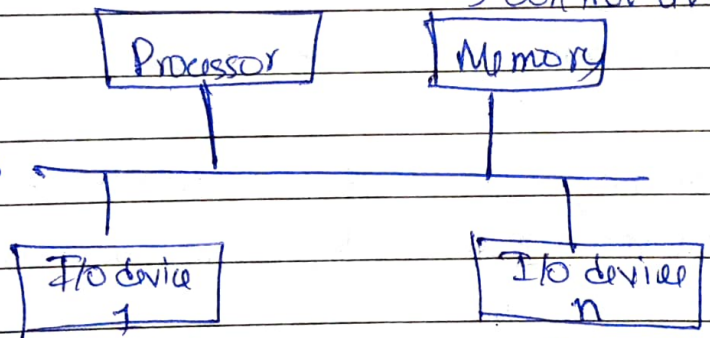
DATE: PAGE:

- Debugger
  - * It helps programmer debug program
  - * It provides trace mode and break points.

- **Debugger**
  - It helps programmer debug program
  - It provides trace mode and break points.

- **Busses**

- They are set of wires used to connect internal parts of a computer
- Only one source/destination pair of units can use this bus to transfer data at any time
- It consists of three lines → Address line
  → Data line
  → Control line




- **Bus Operation**
  - A bus protocol is a set of rules that govern the behavior of various devices connected to the bus, as to when to place information on to the bus and when to assert control signals, etc.

- Address lines → Carry address
  Data lines → Carry data
  Control lines → Carry control signals which specify
  1) If it is read/write operation
  2) Required size of data, when several operand sizes are possible
  3) Timing information to indicate when the processor and I/O device may place data or receive data from bus.

* In data transfer operation, one device plays the role of master. This is the device that initialises data transfer by issueing READ or WRITE command on bus. Usually, it is the processor but even other devices can do the same.

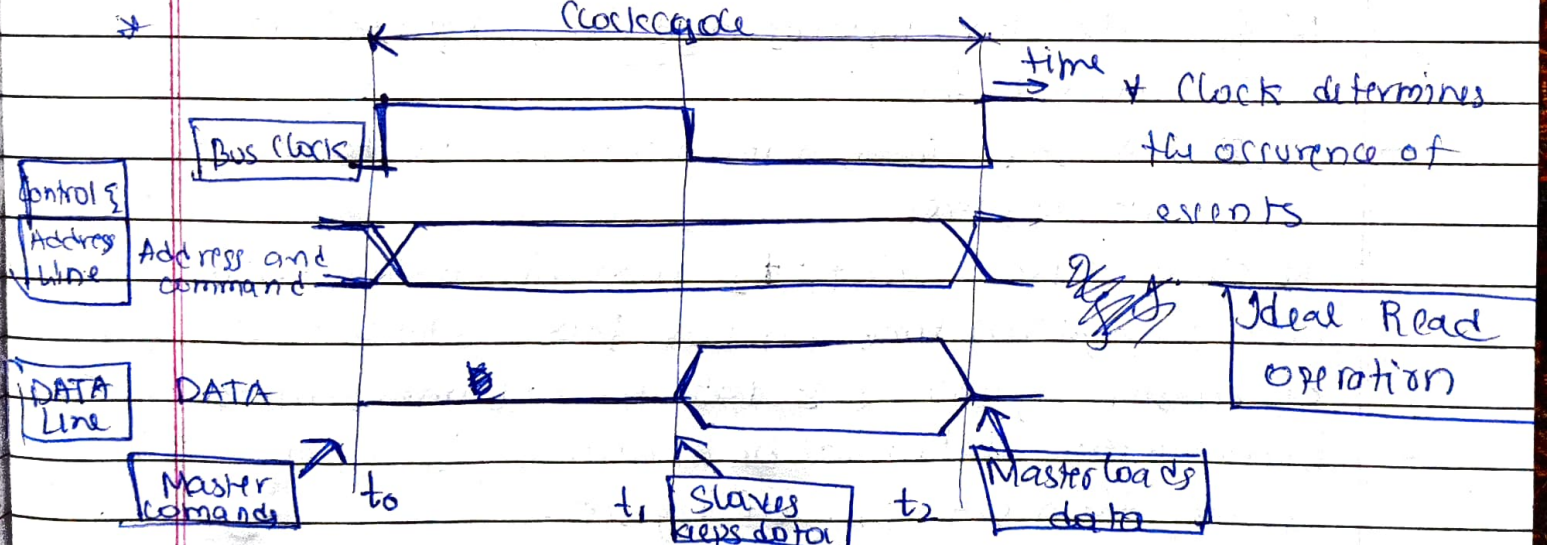
Timing data transfers over bus

- Synchronous
- Asynchronous

Master → Initiates DATA transfer
Slave → Device addressed by master

- Synchronous Bus

* On a synchronous bus, all devices derive timing information from a control line called BUS CLOCK. It has two phases high level followed by a low level. These two phases constitute a clock cycle.
* First half of the cycle b/w two low-to high and high to low transition is called clock pulse.
*




Ideal Timing diagram of input transfer on synchronous bus.

* Clock determines the occurence of events

* Address line and data line are shown as if they are carrying both high and low signals at same time. This is a common convention for indicating that some lines are high and some are low depending on address/data value. The crossing point indicates the time at which these pattern change (at $t_0$ & $t_2$)

* In 1 Bus cycle → 1 data transfer (event)
* Start of clock cycle → Event start to all events start at the beginning of bus cycle / clock cycle.

DATE: PAGE:

* Signal line at level half way b/w the low and high signal level indicates periods during which the signal is unreliable and must be ignored by all devices.

## Read operation

1) At $t_0$ → Master places devices address on address lines and sends command on control line indicating Read operation (with length of operand)

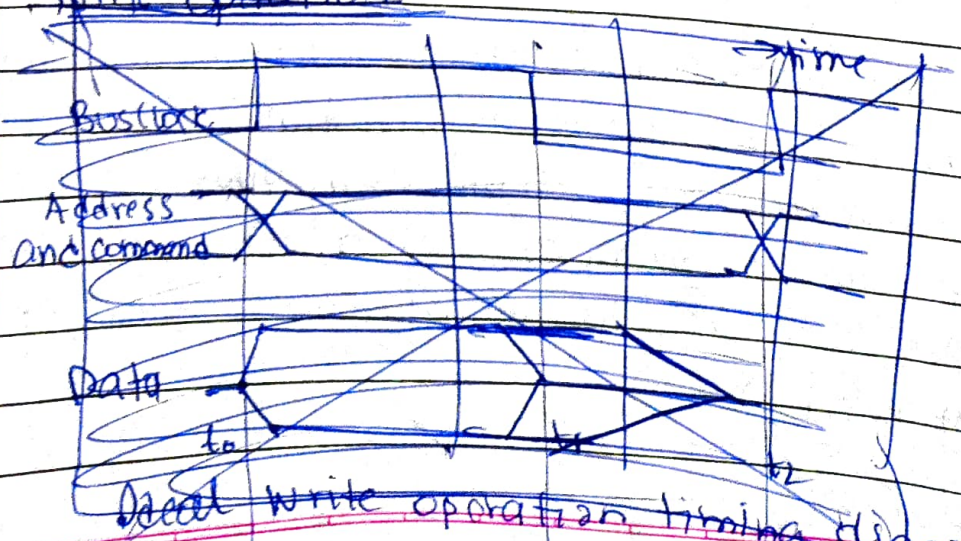
The clock pulse width $t_0$-$t_1$ must be long enough to allow all devices to decode address and control signal so that slave can respond.

2) At $t_1$ → The slave responds and places requested data on data line

3) At $t_2$ → Master loads data from data line to one of its registers.

The clock pulse width $t_1$-$t_2$ must be greater than max propogation time on bus plus setup time of master register for data to be loaded correctly into register, data must be available for a period greater than setup time of register.

## Write operation

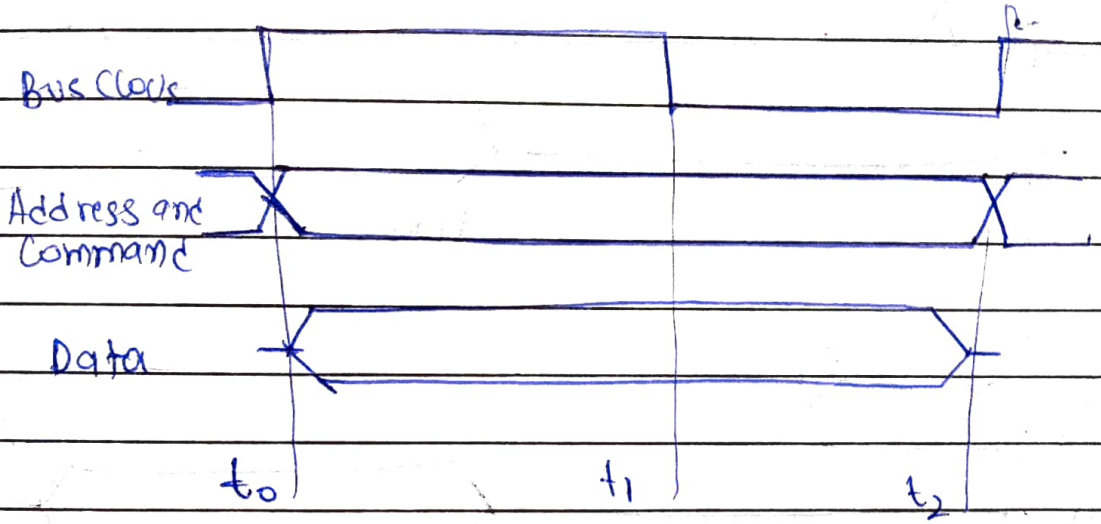



Read write operation timing diagram

## Write operation

DATE: PAGE:

1) At $t_0$ → Master place data on bus along with Write command along with slaves address
2) At $t_2$ → The slave Writes data from data line and puts it into given address.

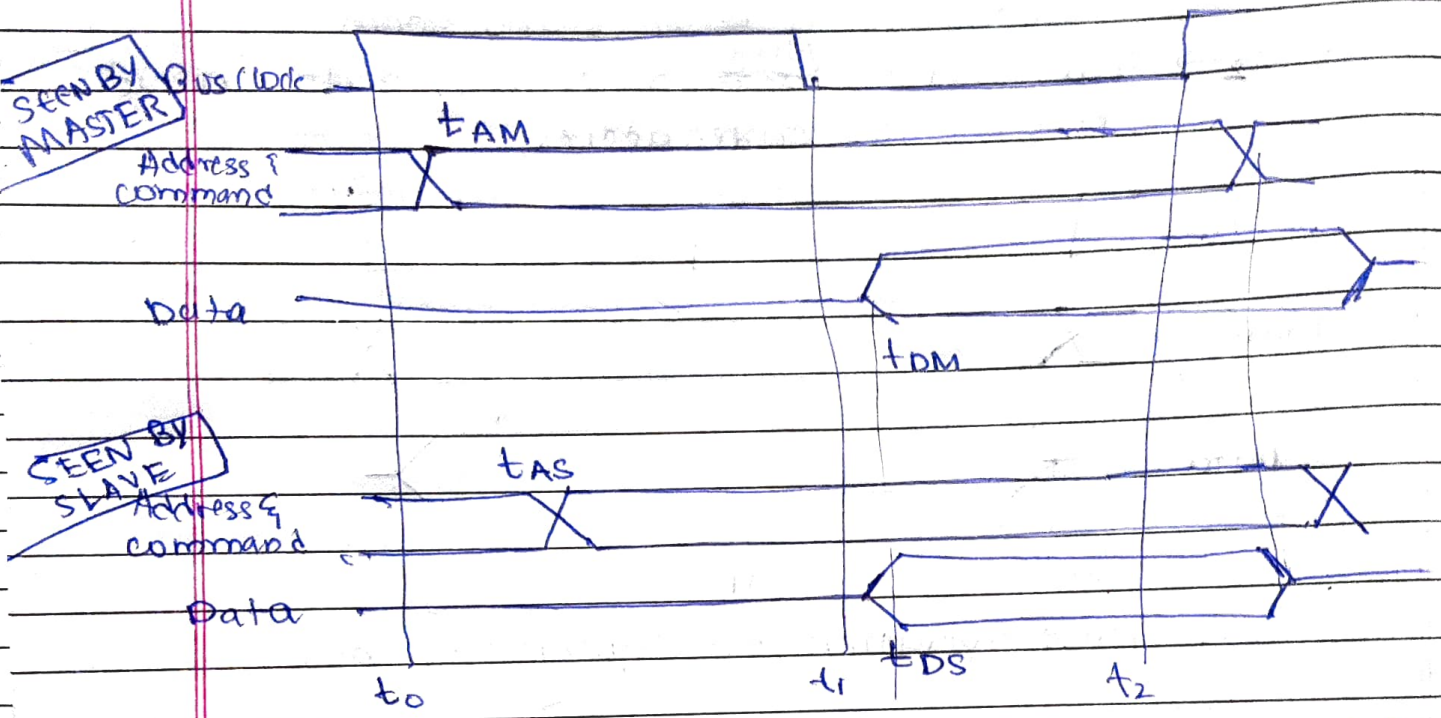



Ideal Write operation on synchronous bus

- These diagrams are ideal because the signals take time to travel from master to slave and this is called as propagation delay.

DATE: PAGE:

Realistic Read operation timing diagram




$t_0 \rightarrow$ The master sends address and command signals at $t_0$. These signals donot actually appear on the bus till $t_{AM}$, due to delay in electronic circuit output from master to bus lines. A short while later at $t_{AS}$, the signals reach slave. The slave decodes the address and at $t_1$ sends the requested data. Here again, the data signal appears on bus at $t_{DS}$. They travel towards master and arrive at $t_{DM}$.

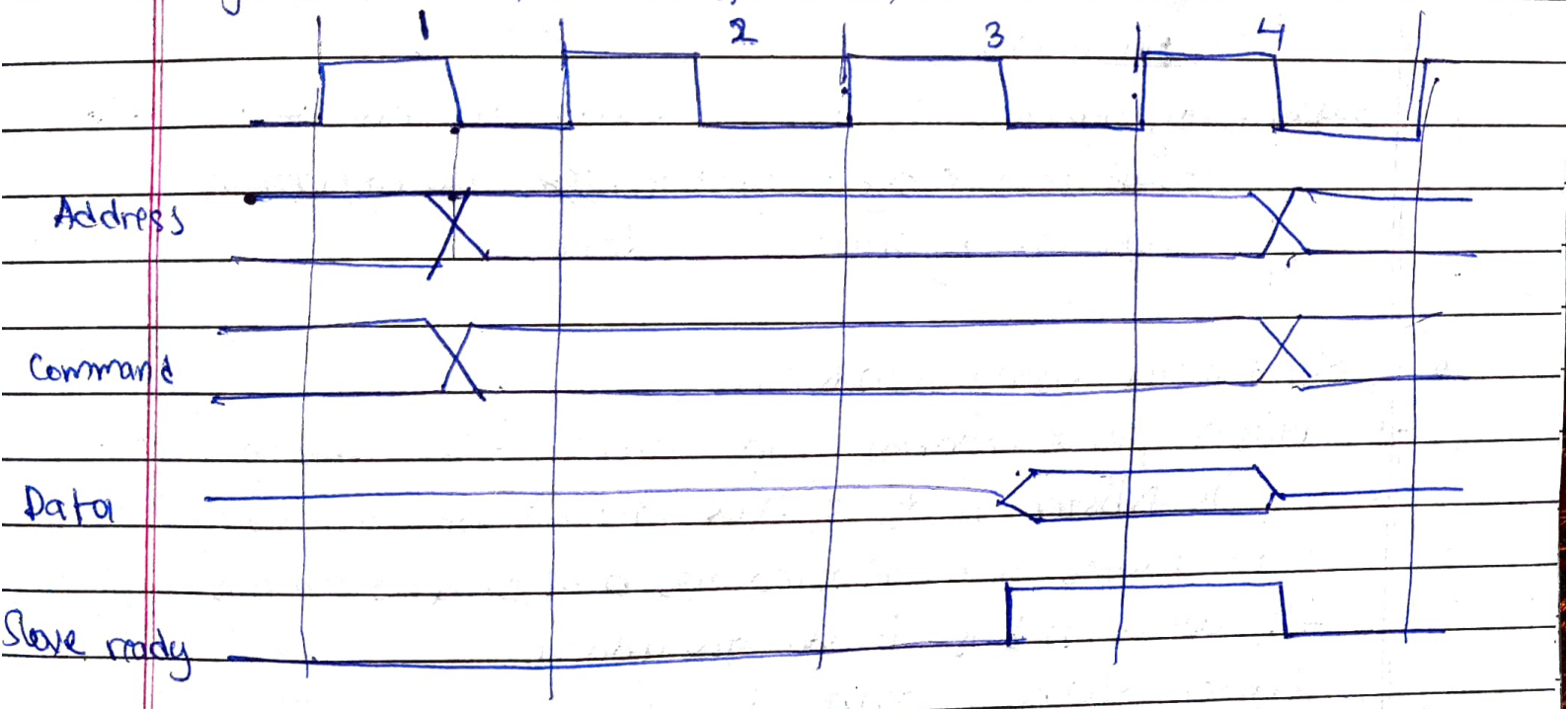
$t_2 \rightarrow$ At $t_2$, master loads data into register. Hence period $t_2 - t_{DM}$ must be greater than setup time of register. The data must be continue to be valid after $t_2$.

Multiple Cycle data transfer

Since transfer has to be completed in one clock cycle this is a limitation, hence time period $t_0 - t_2$ must be chosen to accomodate longest delay

on bus and slowest device in use. This forces the devices to operate at slowest device speed.

* The processor has no info ~~about~~ whether the addressed device has actually responded. At $t_2$, it simply assumes that input data are available on data line in Read operation or the output data have been recieved by the I/O device in Write operation
* To overcome these limitations, most busses incorporate control signals that represent response from the device.




* High frequency bus clock is used
* Clk cycle 1 → Address and command requesting read operation appear on bus
Clk cycle 3 → Slave place data on the bus and asserts slave-ready signal
Clk cycle 4 → Master strobes data into register.

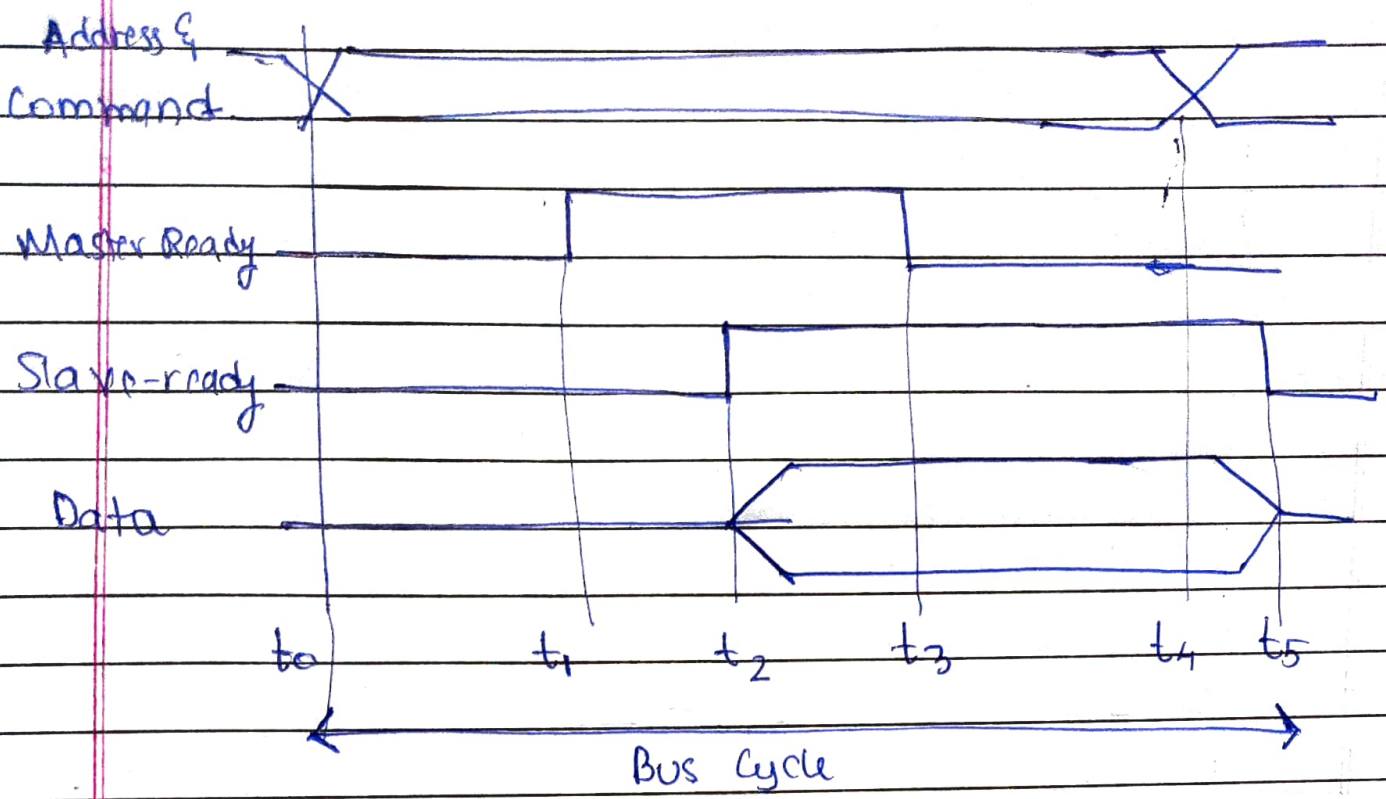
* Here clock changes are seen by all devices at the same time
* Slave ready → Acknowledgement from the slave to the master, confirming the requested data have been placed on the bus.

## Asynchronous Bus

* These don't use clock signals
* Data transfer through these busses are based on use of handshake protocol b/w master and slave. ie exchange of command and response signals b/w master and slave.
* Master-Ready signal → Asserted by master to indicate to the slave that it is ready to participate in data transfer ie Address and commands are placed in bus
* Slave ready signal → Asserted by slave to indicate master that ~~it is ready for data transfer~~. the operation instructed by master is done.

Data transfer using handshake protocol

1. Master places address and command information on bus
2. Asserts Master-ready signal to indicate the slave that address and command has been placed on bus
3. All devices on bus decode the address
4. Addressed slave performs the required operation and informs the master by asserting slave-ready signal and other signals
5. Master removes Master-ready signal once slave ready is asserted
6. If operation is Read operation, Master strobes the data into register.




$t_0$ → Master places address and command information on bus
$t_1$ → Master asserts Master-ready signal
$t_2$ → Addressed slave places the data on the bus and asserts slave-ready signal
$t_3$ → Slave ready signal arrives at master
$t_4$ → Master removes address and command information
$t_5$ → Slave recieves the Master-ready signal transition from 1→0. It removes data and slave ready signal from bus.

- **Arbitration**

* There are occasions when two or more entities contained for use of single resource in a computer. In such cases, it is necessary to decide which divice will access the resource first. This decision is usually made in arbitration process performed by arbitrer circuit

Rest → PPT

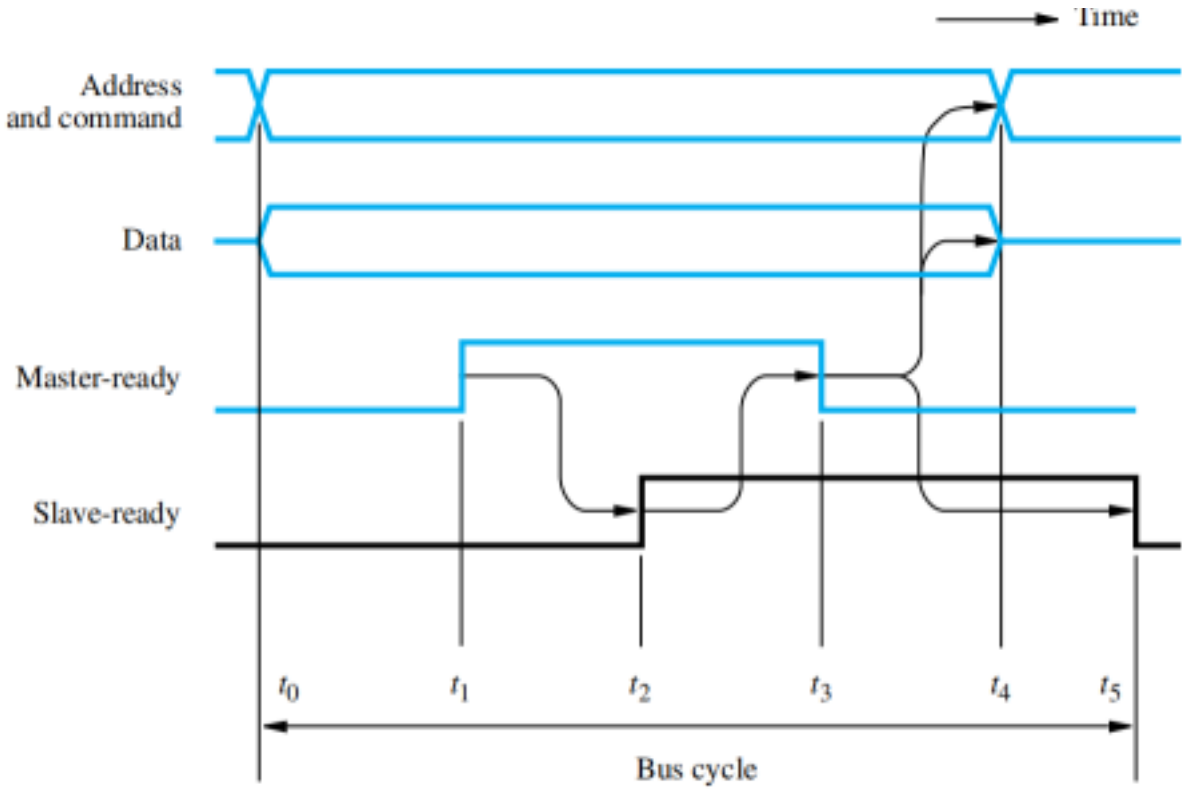



**Figure 7.7** Handshake control of data transfer during an output operation.

# Asynchronous vs. Synchronous bus

- Advantages of asynchronous bus:

- Eliminates the need for synchronization between the sender and the receiver.
- Can accommodate varying delays automatically, using the Slave-ready signal.

## Disadvantages of asynchronous bus:

- Data transfer rate with full handshake is limited by two-round trip delays.
- Data transfers using a synchronous bus involves only one round trip delay, and hence a synchronous bus can achieve faster rates.

# Arbitration

There are occasions when two or more entities contend for the use of a single resource in a computer system. In such cases, it is necessary to decide which device will access the resource first.

The decision is usually made in an arbitration process

performed by an arbiter circuit.

- The arbitration process starts by each device sending a request to use the shared resource.
- The arbiter associates priorities with individual requests. If it receives two requests at the same time, it grants the use of the slave to the device having the higher priority first.

# Arbitration

- A single bus is the shared resource.
- The device that initiates data transfer requests on the bus is the bus master. In case of multiple requests for the shared bus at the same time the decision to select next master is made by an arbiter circuit.
- The arbitration process starts by each device sending a request to use the shared resource.
- The arbiter associates priorities with individual requests. If it receives two requests at the same time, it grants the use of the slave to the

device having the higher priority first.

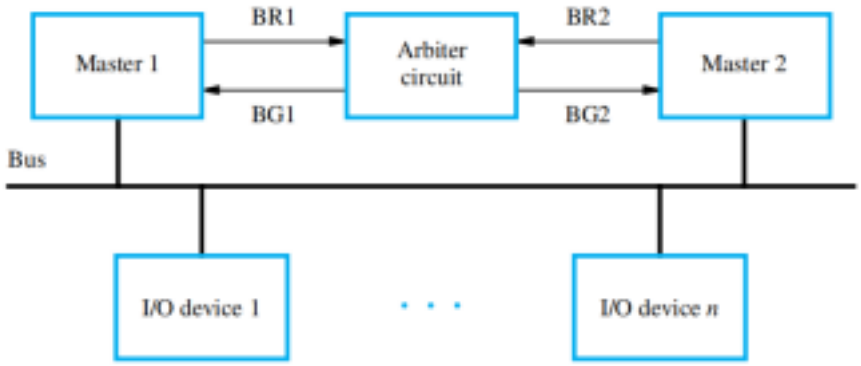



**Figure 7.8** Bus arbitration.

# Arbitration

- Assume that master 1 has the highest priority, followed by the others in increasing numerical order.
- Master 2 sends a request to use the bus first, the arbiter grants the bus to this master by asserting BG2.
- When master 2 completes its data transfer operation, it releases the bus by deactivating BR2. By that time, both masters 1 and 3 have activated their request lines.

🗣 Note that the bus is granted to master 1 before master 3 even though master 3 activated its request line before master 1.

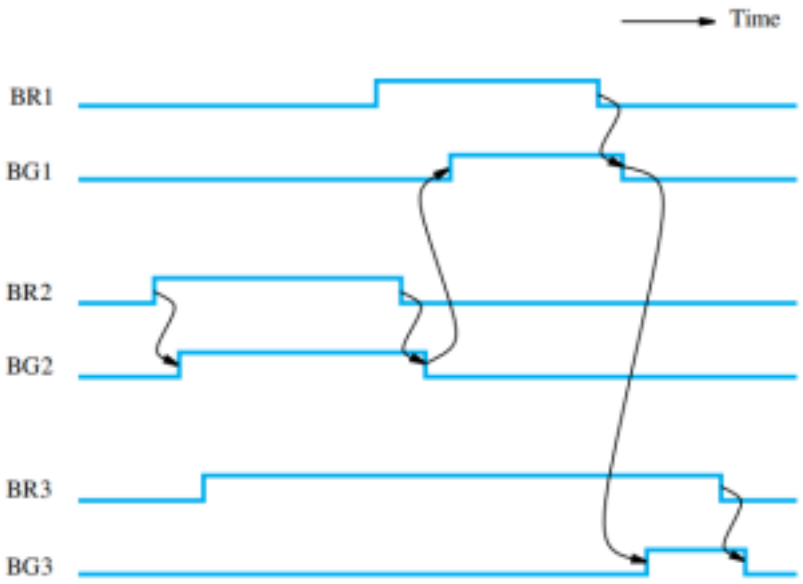



**Figure 7.9** Granting use of the bus based on priorities.



# Interface circuits

🗣 I/O interface consists of the circuitry required to